Raspberry Pi Password Cracking Lab (v.1.0.0)

Objective

Develop a Python application on a Raspberry Pi that demonstrates how a dictionary (wordlist) attack works by comparing entries from the wordlist against a user-provided password. The application displays each password attempt in real time until a match is found or the wordlist is exhausted.

Overview

Passwords are one of the most common authentication methods used today. If users chose weak or commonly used passwords, they may be vulnerable to dictionary attacks. This project demonstrates how attackers can automate password guessing using publicly available password lists such as in demo-wordlist-1k.txt. This application serves as an educational tool to visualize how quickly weak passwords can be discovered.

Prerequisites

- Raspberry Pi 4 Model B / Other Raspberry Pi Model (Optional)
- Raspberry Pi Desktop (Bullseye), based on Debian GNU/Linux 11 (ISO)
- Python 3.x
- demo-wordlist-1k.txt
- Keyboard and mouse
- Monitor connected to the Raspberry Pi
- John The Ripper
- OpenSSL

Note: This guide assumes Raspberry Pi OS has already been installed and configured. Installation of the operating system is outside the scope of this document.

Installation

1. Update the Operating System

Before installing the project, ensure that the system packages are up to date. You will need root privileges to update your system if your user isn't in the sudoers file. Type:

- **su –**

Then, enter your root password. Then run this command:

- **sudo apt update && sudo apt upgrade**

2. Verify Python 3 Installation
    - **python3 –version**

If Python 3 is not installed, install it using:

- **sudo apt install python3**

3. Verify OpenSSL Installation by using:
    - **openssl version**

If OpenSSL is not installed, install it by using:

- **sudo apt install openssl**

4. Install John the Ripper.
    - **sudo apt install john**

Verify the installation by running

- **john**

The terminal should display john the ripper version and available command line options.

5. Clone the repository

Download the project files

- **git clone** https://github.com.Aria-C900/raspberry-pi-demo-dictionary-attack-lab.git
- **cd raspberry-pi-demo-dictionary-attack-lab**

Verify the project files were downloaded successfully by typing:

- **ls**

You should see the files named "**test_crack.py**", "**demo-wordlist-1k.txt**", "**README.MD**" and "**LICENSE**" in the repository.

6. Create a folder in **/usr/share** named **wordlists** and move **demo-wordlist-1k.txt** into the folder. You can do this by typing this command:
    - **cd /home/username/usr/share**

Then create the folder by typing:

- **mkdir wordlists**

7. You will need to be in the correct directory where demo-wordlist-1k.txt is currently hosted. Type this command until you reach /home/username again.
    - **cd ..**

Enter back into **raspberry-pi-demo-dictionary-attack-lab** directory, and then type this command:

- **mv demo-wordlist-1k.txt /usr/share/wordlists**

Your result should be **/usr/share/wordslists/demo-wordlist-1k.txt.** You can check it with:

- **ls /usr/share/wordlists**

***Note: By default, this project expects the demonstration wordlist to be in /usr/share/wordlists/demo-wordlist-1k.txt. This is not required. You can choose to put the wordlist elsewhere but ensure that the script reflects that. For example:**

- WORDLIST = "/path/to/your/file/demo-wordlist-1k.txt"

8. Run the Program

Navigate back to raspberry-pi-demo-dictionary-attack-lab directory and type:

- **chmod +x test_crack.py**

Then run this command:

- **python3 test_crack.py**

The program should begin to run in the terminal. If you wish to exit before it finishes, use the shortcut CTRL + X. Otherwise, exit by pressing "n" when prompted.

Project Structure

This project consists of a small number of files required to demonstrate a dictionary attack using a demo wordlist. These are the main files you need to run the program.

pi-dashboard-demo/

|— test_crack.py

|— demo-wordlist-1k.txt

|— README.md

|--- LICENSE

File Descriptions

test_crack.py → This is the primary python application. It will prompt the user for a password, read entries from demo-wordlist-1k.txt, compares each password in the wordlist until a match is found or the wordlist is exhausted.

demo-wordlist-1k.txt → a password dictionary containing millions for the used passwords. For the sake of the demonstration, it has been cut down to 3,000 words.

README.md → contains documentation and instructions for installing and running the project.

LICENSE → Contains the MIT License for this project. The license permits others to use, study, modify, and redistribute the project's source code, provided the original copyright notice and license are retained.

Usage

Once the program has been installed and the required files are present, the application can be launched from the project directory.

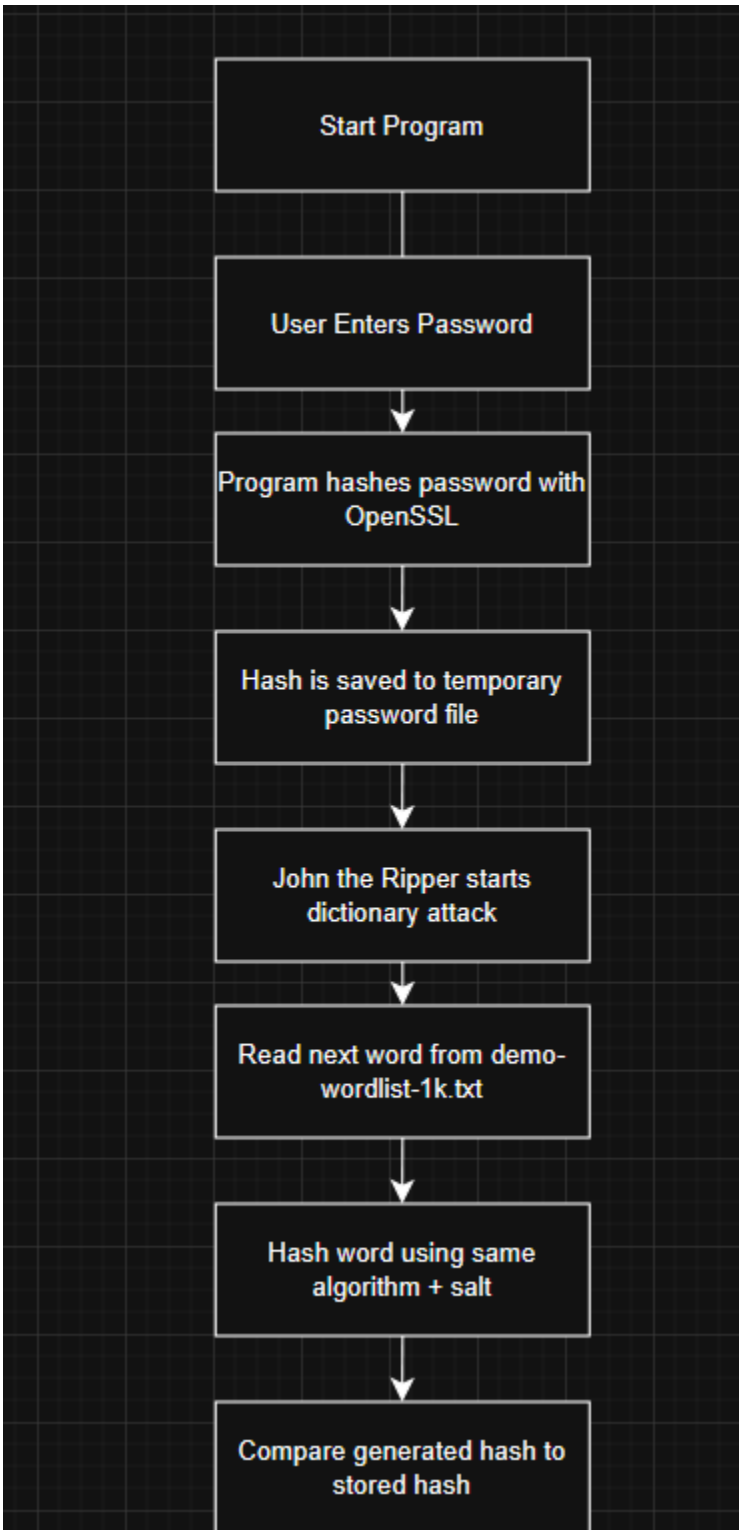
- **python3 test_crack.py**

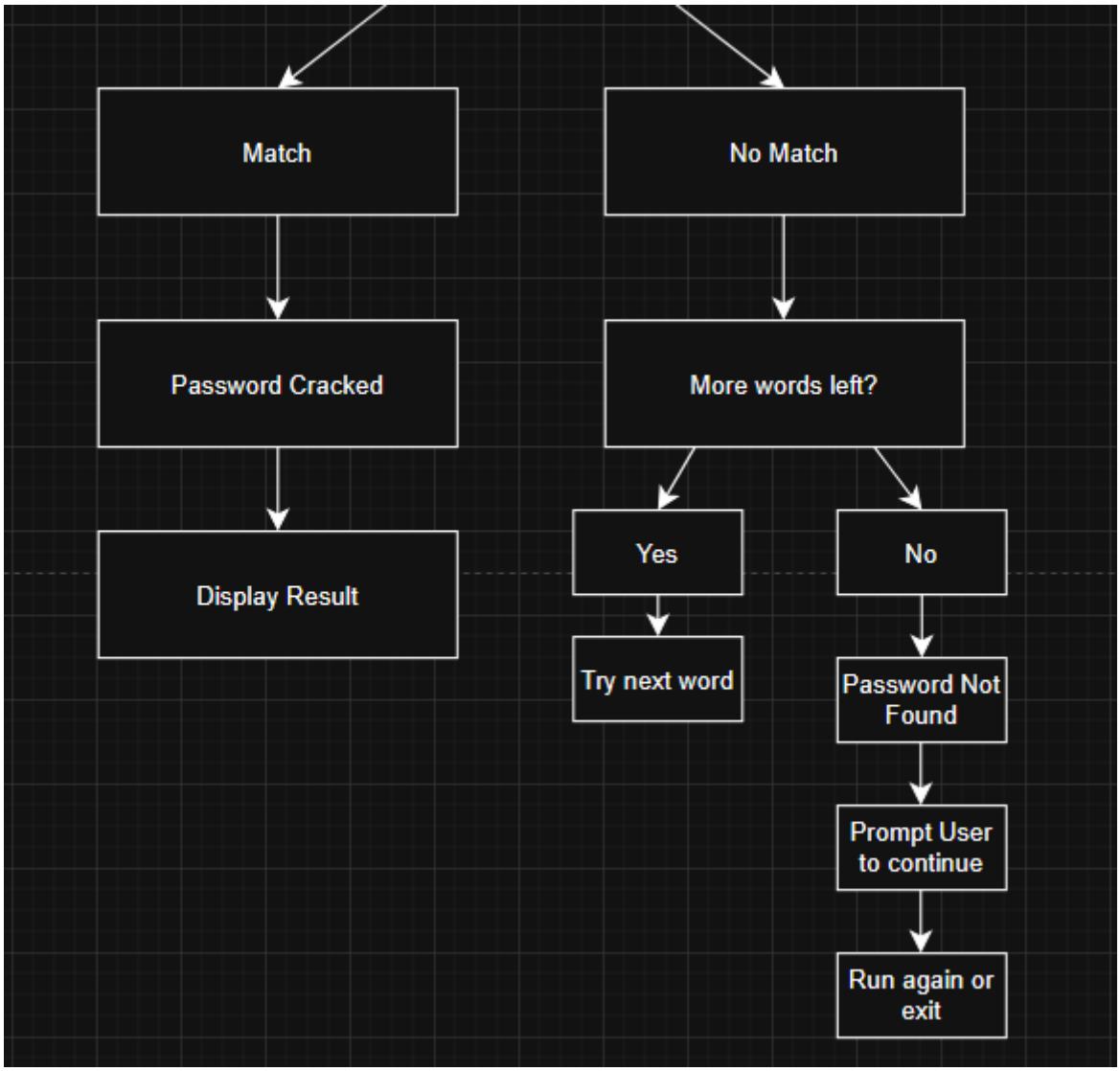
After the program launches, it will prompt the user to enter a password.

1. Enter a password and press Enter.
2. The application hashes the password using OpenSSL and stores the hash in a temporary password file.
3. John the Ripper begins a dictionary attack by using demo-wordlist-1k.txt, displaying a “Cracking…” loading animation while attempting to identify the password.
4. If the password is found within the wordlist, the application displays a success message showing the recovered password and prompts the user to continue (y/n).
5. If John the Ripper reaches the end of the wordlist without finding a matching password, the application notifies the user that the password could not be found and prompts the user to continue (y/n).
6. If the user enters (n), the program exits. Entering (y) starts another-password cracking demonstration.

Note: This application is intended for educational purposes only. It demonstrates how offline dictionary attacks function by comparing a user-provided test password against entries in a password wordlist.

How the Program Works





Script Function

import subprocess

➔ Allows Python to run external commands like **openssl** and **john the Ripper**.

import os

➔ Lets Python interact with files, like deleting TEMPFILE

import getpass

➔ Lets the users type a password without showing it on screen

import time

➔ Used to load the animation delay

Main Variables

WORDLIST = “/path/to/your/directory/demo-wordlist-1k.txt"

SALT = "club123"

PASSFILE = "student_passwd.txt"

TEMPFILE = "temp_passwd.txt"

WORDLIST stores the file path to the password wordlist that John the Ripper will be using dring the dictionary attack. This path must match the actual location of demo-wordlist-1k.txt or the program will not run correctly.

SALT is used for OpenSSL to hash the user’s input password. The same salt must be sued when John hashes candidate passwords from the wordlist so the generated hashes can be compared correctly.

PASSFILE and **TEMPFILE** are variables that define the files where the generated hash is written. The program automatically creates these files when run, so the user does not need to manually create them beforehand.

Loading Animation

```
def loading_animation(text="Cracking", duration=3):
  print()
  for i in range(duration):
    print(f"\r{text}{'.' * (i % 4)}", end="", flush=True)
    time.sleep(0.6)
  print("\n")
```

The **loading_animation()** function provides a visual indicator that the application is processing data. The function accepts two parameters, **text** and **duration**. **Duration** represents the number of animation cycles to display before continuing with the remainder

of the program. The **for** loop controls how many times the animation updates, while **time.sleep(0.6)** pauses the program for 0.6 seconds between each update. The **\r** (carriage return) character allows the program to overwrite the current line instead of printing a new line each time.

Run_cracker() Function

```
def run_cracker():
    print("Welcome to the Password Cracker Demo!")
    print("For educational use only. Please don't use real passwords.")
    password = getpass.getpass("Enter a password: ")
```

The **run_cracker()** function controls a single execution of the password-cracking demonstration. It is responsible for guiding the user through the process of entering a password, generating its hash, performing the dictionary attack and displaying the results.

The first two **print()** statements display a welcome statement and remind the user that the application is intended for educational purposes only.

The program prompts the user to enter a password while using **getpass.getpass()** This function hides the characters typed by the user.

Password Hashing and Temporary File Creation

```
    print("\n Hashing your password...")
    hashed_pw = subprocess.check_output(["openssl", "passwd", "-1", "-salt", SALT, password]).decode().strip()
```

After the user inputs a password, and hits enter, the application displays a message indicating that the password is being hashed. Rather than storing the password in plaintext, the program uses OpenSSL to generate a cryptographic hash of the user's input.

The subprcess.check_output() function executes the external OpenSSL command from within python. The -1 option instructs OpenSSL to generate an MD5-crypt hash, while the -salt option specifies the salt value defined earlier in the program (club123). The resulting hash is decoded from byte into a string and stored in the hashed_pw variable.

Writing the Password Hash

```
with open(PASSFILE, "w") as f:
    f.write(f"student:{hashed_pw}\n")
with open(TEMPFILE, "w") as tf:
    tf.write(f"student:{hashed_pw}\n")
```

Once the password has been hashed in the program, the program generates two temporary text files. Each file stores the username (student) followed by the generated password hash in the format expected by John the Ripper.

PASSFILE serves as a record of the generated password hash, while **TEMPFILE** is used as the input file that John attempts to crack during the dictionary attack. Separating these files allows the demonstration to preserve the generated hash while safely deleting the temporary cracking file after the attack is complete.

Beginning the Dictionary Attack

```
print("Attempting to crack it with a common wordlist...")
loading_animation("Cracking", 5)
```

The **loading_animation()** functions runs to indicate progress as the cracking process is initiated.

Dictionary Attack Execution

```
result = subprocess.run(
  ["john", f"--wordlist={WORDLIST}", TEMPFILE],
  stdout=subprocess.PIPE,
  stderr=subprocess.PIPE
)
```

After the password hash has been saved to the temporary password file, John the Ripper is launched using **subprocess.run()** function. This allows the application to execute an external command line program while still in the Python script.

John is given two bits of information, specifically **–wordlist={WORDLIST}** and **TEMPFILE**.

–wordlist={WORDLIST} specifies the location of the demonstration password dictionary**.**

TEMPFILE contains the password hash generated earlier in the program.

Now, John the Ripper can begin the offline dictionary attack. It reads each candidate password from the wordlist, hashes it with the same hashing algorithm and salt, and compares the generated hash against the stored hash in **TEMPFILE**. This process repeats until either a matching hash is found or every password in the wordlist has been tested.

The program has the standard output (**stdout**) and errors to pipes (**stderr)**, leaving out John the Ripper's console output.

Displaying the Result

```
output = subprocess.check_output(["john", "--show", TEMPFILE]).decode()
```

After John the Ripper completes the dictionary attack, the program will execute a second command using **john –show**. This command displays passwords recovered during the attack. The output is captured to **subprocess.check_output()**, converted from bytes into a standard Python string using **.decode()** and stored in the variable **output.**

Processing the Results

```
line = next((ln for ln in output.splitlines() if ln.startswith("student:")), None)
```

After **john –show** returns its output, the program searches for an entry beginning with **"student:"**. This is the username that was written to the temporary password file earlier in the program.

The **splitlines**() function separates the command output into individual lines, while the **next()** function returns the first line matching the specified condition. If no matching line is found, **None** is returned instead.

Displaying the Password

```
if line:
    cracked_pw = line.split(":", 1)[1]
    print(f"Cracked! Password was: {cracked_pw}")
else:
    print("Too strong! This password wasn't in the list.")
```

Now, the program needs to determine whether John the Ripper successfully recovered the password.

If a matching line exists, the application separates the username from the recovered password using the **split()** function. The recovered password is stored in the variable **cracked_pw** and displayed to the user with a success message.

If no matching line is returned, the program concludes that the password was not present within the demonstration wordlist and displays a message informing the user that the password could not be recovered.

Cleanup and User Interaction

```
try:
    os.remove(TEMPFILE)
except FileNotFoundError:
    pass

input("\n Press Enter to try another password.")
```

The program will remove the temporary password file used during the cracking process. The **os.remove() function deletes TEMPFILE**, ensuring that temporary files created during execution are not left behind after the program finishes.

The file deletion is within a **try/except** block – meaning that if the file is already removed or cannot be found, a **FileNotFoundError** will occur, and python will ignore this statement by using the **pass** statement, so that the program can run normally without interruption.

Program Loop

```
while True:
  run_cracker()
  again = input("\nTry another password? y/n: ").lower()
  if again != "y":
    print("Goodbye!")
    break
```

This is the end of the program and a while True statement that allows **run_cracker()** function to loop continuously until the user exits. Each iteration calls the **run_cracker()** function, which runs the entire program. After the demonstration is completed, the user is

prompted on whether they would like to continue the program. The **.lower()** method converts the user's response to lowercase, allowing inputs such as "**Y**" and "**y**" to be treated as the same.

If the user enters anything that is not "**n**", the program will print a "Goodbye!" message and execute the **break** statement, terminating the program.

Troubleshooting

If you are having issues running this program, verify the following:

- Ensure that you have python3 or newer installed by running **python3 –**version.
- Ensure that you have OpenSSL installed by running **openssl version**.
- Ensure that you have John the Ripper installed by running **john**.
- Confirm that the demonstration wordlist (demo-wordlist-1k.txt) exists and that the **WORDLIST** variable in **test_crack.py** points to the correct file path.
- Confirm you are executing the program from the project directory.
- Verify that your version of **John the Ripper** supports the hash format specified in the script. Older versions may only support **MD5-crypt (-1)** while newer versions support stronger hash formats such as **SHA-512 crypt (-6).**
- If using Linux, you will need to run **chmod +x** to run the python script before using **python3 <program.py>** to execute it.
- If the application reports that a password cannot be cracked, verify that the password exists within **demo-wordlist-1k.txt**.

If issues persist, notify the instructor for further assistance.

Feel free to experiment with the program by creating your own copy of the project and modifying the code. Try using different wordlists, changing the interface, or adding new features to gain a better understanding of Python programming and dictionary attacks.

Limitations

- The program performs a offline dictionary attack using the demo-wordlist-1k.txt and can only discover passwords that exist within that file.
- Passwords are tested sequentially, one entry at a time. The application does not implement optimizations.
- The application was developed and tested Raspberry Pi OS (Bullseye). Additional configuration may be required to run the graphical interface on other Linux distributions or operating systems.

- Older versions of John the Ripper may only support **MD5-crypt (-1)**. Newer hash formats, such as **SHA-512 crypt (-6)**, may require a newer version of John the Ripper.
- Real world password auditing tools use much larger wordlists containing billions of passwords, which increases the likelihood of recovering week passwords.
- The project is intended for educational purposes only and is not designed nor intended for use against real systems or accounts.

Future Improvements

- Support password hashes (MD5, SHA-256, NTLM, bcrypt)
- Allow users to select different wordlists
- Display progress bars and estimated completion time
- Add multithreading to improve performance
- Improve cross-platform compatibility (Windows, macOS)
- Record statistics such as elapsed time and number of attempts
- Support multiple attack modes to compare the strengths and weaknesses of each password-cracking technique